

Analisis Kinerja Regresi dan Klasifikasi Model Hybrid CNN-LSTM Ringkas untuk Prediksi State of Charge Baterai

Regan Agam

Departemen Teknik Elektro dan Teknologi Informasi
Universitas Gadjah Mada
Yogyakarta, Indonesia
reganagam@mail.ugm.ac.id

Ringkasan

Estimasi State of Charge (SoC) yang akurat adalah pilar fundamental dalam sistem manajemen baterai (BMS) untuk kendaraan listrik (EV). Meskipun model deep learning seperti CNN dan LSTM telah terbukti efektif, arsitektur yang kompleks seringkali menghambat implementasi praktis karena tuntutan komputasi yang tinggi. Penelitian ini mengusulkan dan mengevaluasi secara komprehensif sebuah model hybrid CNN-LSTM yang ringkas untuk prediksi SoC. Dengan menggunakan dataset primer berukuran besar, model dievaluasi tidak hanya sebagai tugas regresi untuk prediksi nilai kontinu, tetapi juga sebagai tugas klasifikasi untuk menentukan rentang SoC. Hasil evaluasi regresi menunjukkan performa luar biasa dengan Mean Squared Error (MSE) sebesar 0.0124 dan R-squared (R^2) 0.9870. Lebih lanjut, evaluasi klasifikasi (dengan 5 interval SoC) menghasilkan akurasi dan F1-Score yang tinggi. Performa ganda ini, yang dicapai dengan arsitektur yang hanya memiliki 23.415 parameter, membuktikan bahwa model yang diusulkan tidak hanya akurat dalam memprediksi nilai SoC, tetapi juga andal dalam mengkategorikan kondisi baterai, menjadikannya solusi yang efisien, kuat, dan praktis untuk aplikasi BMS di dunia nyata.

Index Terms

State of Charge (SoC), Baterai Lithium-ion, Deep Learning, CNN-LSTM, Regresi, Klasifikasi, Sistem Manajemen Baterai (BMS).

I. PENDAHULUAN

Transisi global menuju elektrifikasi transportasi menempatkan teknologi baterai sebagai pusat inovasi. Sistem Manajemen Baterai (BMS) memegang peranan vital dalam memastikan operasi baterai yang aman, andal, dan efisien. Di antara berbagai parameter yang dipantau, State of Charge (SoC)—yang mengindikasikan sisa energi baterai—merupakan yang paling krusial bagi pengguna [1]. Estimasi SoC yang presisi dapat mencegah kerusakan baterai akibat *over-charging* atau *over-discharging* dan memberikan informasi jarak tempuh yang akurat kepada pengemudi.

Metode estimasi SoC tradisional memiliki keterbatasan signifikan [2]. Sebagai alternatif, metode berbasis data yang memanfaatkan deep learning telah menunjukkan kemajuan pesat. Model-model seperti Long Short-Term Memory (LSTM) [5] dan Convolutional Neural Networks (CNN) [3], [4], [6] mampu memodelkan hubungan non-linear yang kompleks antara parameter input (arus, tegangan, suhu) dan SoC.

Namun, tren saat ini seringkali mengarah pada arsitektur yang sangat dalam dan kompleks, seperti model Transformer [7] atau Bi-LSTM bertumpuk [8], yang memiliki jutaan parameter. Meskipun akurat, model-model ini menuntut sumber daya komputasi yang besar dan waktu pelatihan yang lama, menjadikannya tidak praktis untuk diimplementasikan pada perangkat keras BMS yang umumnya memiliki keterbatasan komputasi.

Mengatasi tantangan ini, penelitian ini berfokus pada pengembangan dan analisis komprehensif sebuah arsitektur hybrid CNN-LSTM yang ringkas dan efisien. Kontribusi utama dari paper ini adalah:

- **Analisis Kinerja Ganda:** Mengevaluasi model tidak hanya sebagai masalah regresi (memprediksi nilai SoC yang tepat) tetapi juga sebagai masalah klasifikasi (mengidentifikasi rentang SoC), yang memberikan wawasan lebih praktis untuk aplikasi BMS.
- **Efisiensi Komputasi:** Mendemonstrasikan bahwa arsitektur yang ringan dengan hanya 23.415 parameter dapat mencapai kinerja canggih.
- **Validasi pada Data Primer:** Menggunakan dataset primer berukuran besar untuk memastikan relevansi dan robustitas model dalam skenario dunia nyata.

II. METODOLOGI PENELITIAN

A. Deskripsi dan Prapemrosesan Dataset

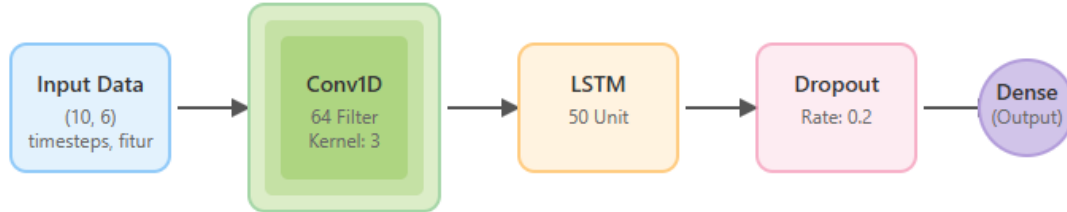
Data yang digunakan dalam penelitian ini adalah dataset primer time-series yang diperoleh dari salah satu perusahaan di Indonesia, berisi rekaman parameter operasional baterai. Untuk pemodelan, dipilih 6 fitur input yang paling relevan: **tegangan**, **arus**, dan empat pengukuran **suhu** (temp 1-4). Variabel target adalah **SoC**.

Langkah-langkah prapemrosesan data meliputi:

- 1) **Normalisasi:** Semua fitur input dan target dinormalisasi ke rentang $[0, 1]$ menggunakan MinMaxScaler.
- 2) **Pembuatan Urutan (Sequence Generation):** Data diubah menjadi sekuens-sekuens yang tumpang tindih. Setiap sekuens input terdiri dari data selama 10 langkah waktu (*timesteps*) terakhir untuk memprediksi nilai SoC pada langkah waktu berikutnya.
- 3) **Pembagian Data:** Dataset dibagi secara acak menjadi data latih (70%), data validasi (20%), dan data uji (10%).

B. Arsitektur Model yang Diusulkan

Model yang diusulkan mengintegrasikan lapisan Conv1D untuk ekstraksi fitur spasial dari data sekuensial dan lapisan LSTM untuk menangkap dependensi temporal. Arsitektur ini dirancang secara spesifik agar efisien, dengan total hanya 23.415 parameter.



Gambar 1. Arsitektur model hybrid CNN-LSTM yang diusulkan.

Struktur model (Gambar 1) adalah sebagai berikut:

- 1) **Input Layer:** Menerima sekuens data berdimensi (10, 6).
- 2) **Conv1D Layer:** 64 filter dengan kernel size 3 dan aktivasi ReLU.
- 3) **LSTM Layer:** 50 unit dengan aktivasi ReLU.
- 4) **Dropout Layer:** Rate 0.2 untuk regularisasi.
- 5) **Dense Layer:** Satu neuron output untuk prediksi nilai SoC.

C. Pengaturan Eksperimen dan Metrik Evaluasi

Model dilatih menggunakan optimizer Adam dan fungsi kerugian MSE. Kinerja model dievaluasi dari dua perspektif:

- 1) **Evaluasi Regresi:** Untuk mengukur akurasi prediksi nilai kontinu SoC, digunakan metrik: Mean Squared Error (MSE) dan R-squared (R^2).
- 2) **Evaluasi Klasifikasi:** Untuk menilai kemampuan model dalam mengkategorikan rentang SoC, nilai SoC kontinu dikonversi menjadi 5 kelas interval. Kinerja dievaluasi menggunakan Akurasi, Presisi, Recall, F1-Score, dan divisualisasikan dengan *Confusion Matrix*.

III. HASIL DAN PEMBAHASAN

Model yang telah dilatih dievaluasi secara menyeluruh pada data uji.

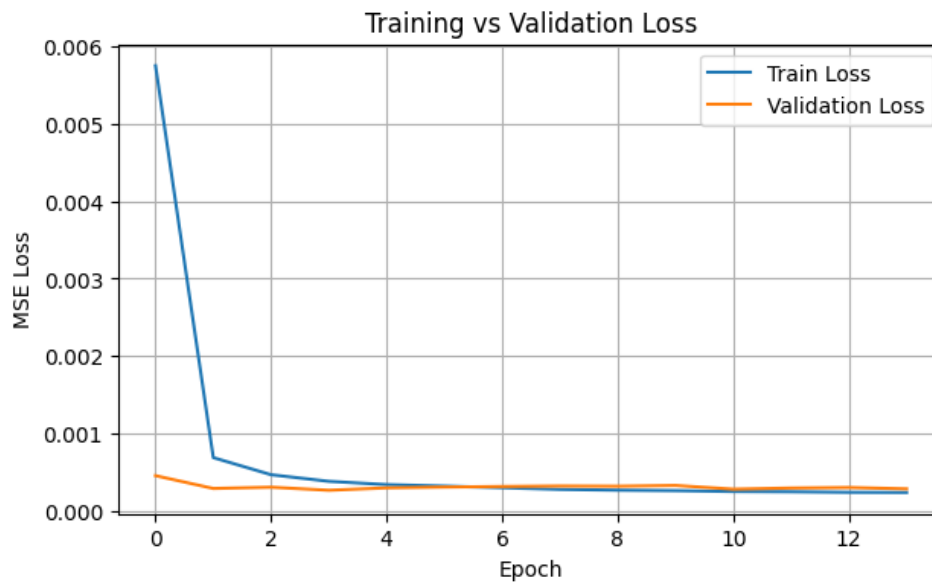
A. Kinerja sebagai Model Regresi

Tabel I merangkum metrik kinerja model dalam memprediksi nilai SoC yang kontinu.

Tabel I
HASIL KINERJA REGRESI PADA DATA UJI

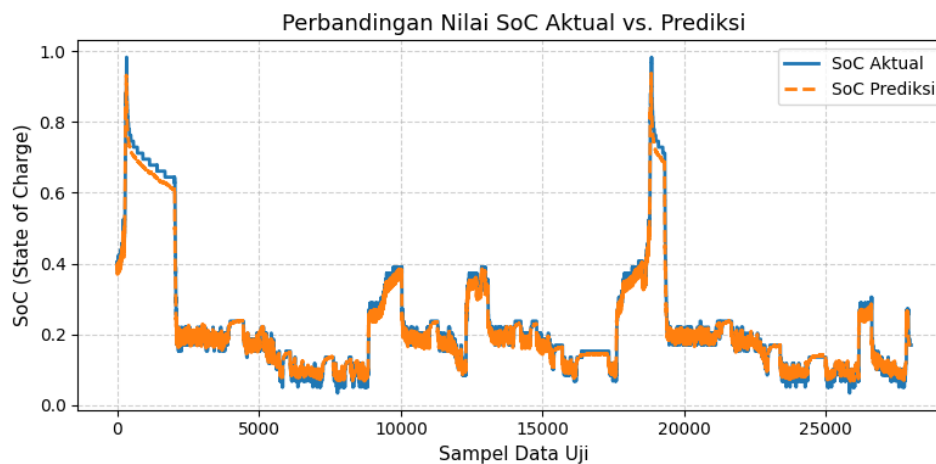
Metrik	Nilai
Mean Squared Error (MSE)	0.0124
R-squared (R^2)	0.9870

Hasil regresi menunjukkan tingkat akurasi yang sangat tinggi. Nilai R^2 yang mendekati 1 (0.9870) menegaskan bahwa model mampu menjelaskan hampir semua variabilitas dalam data SoC, menandakan kecocokan model yang luar biasa. Visualisasi dari hasil ini dapat dilihat pada grafik-grafik berikut.



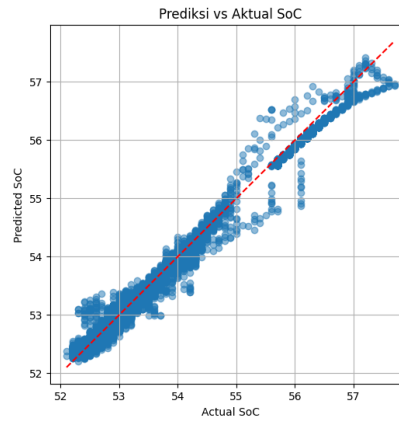
Gambar 2. Grafik Training & Validation Loss. Konvergensi yang stabil menunjukkan model tidak mengalami overfitting.

Analisis kurva pada Gambar 2 menunjukkan proses pelatihan yang sehat. Kedua kurva, baik *training loss* (kerugian pada data latih) maupun *validation loss* (kerugian pada data validasi), sama-sama menurun secara signifikan pada epoch-epoch awal dan kemudian mencapai konvergensi pada nilai yang sangat rendah. Hal yang terpenting adalah kurva *validation loss* bergerak secara paralel dengan *training loss* tanpa menunjukkan tanda-tanda peningkatan. Ini adalah indikator kuat bahwa model memiliki kemampuan generalisasi yang baik dan tidak mengalami *overfitting*, di mana model hanya menghafal data latih tetapi gagal berkinerja baik pada data baru.



Gambar 3. Perbandingan Nilai SoC Aktual vs. Prediksi. Kurva prediksi hampir sepenuhnya menutupi kurva aktual.

Gambar 3 menyajikan visualisasi paling intuitif dari akurasi model dalam domain waktu. Grafik ini memplot nilai SoC aktual (biru) dan nilai SoC yang diprediksi oleh model (oranye) untuk setiap sampel pada data uji. Terlihat jelas bahwa kurva prediksi hampir berimpit sempurna dengan kurva aktual. Ini menunjukkan bahwa model tidak hanya akurat dalam memprediksi nilai titik individu, tetapi juga sangat mampu menangkap dinamika temporal dari data, termasuk tren pengisian (*charging*), pengosongan (*discharging*), serta perubahan-perubahan kecil yang terjadi. Kemampuan melacak profil SoC secara dinamis ini sangat krusial untuk aplikasi BMS di dunia nyata.



Gambar 4. Scatter Plot Prediksi vs. Aktual SoC. Titik-titik data yang terkonsentrasi di sepanjang garis diagonal menunjukkan korelasi yang hampir sempurna.

Untuk mengonfirmasi konsistensi prediksi di seluruh rentang nilai SoC, analisis *scatter plot* (Gambar 4) dilakukan. Setiap titik pada plot ini merepresentasikan satu sampel data uji, dengan sumbu-x sebagai nilai SoC aktual dan sumbu-y sebagai nilai SoC prediksi. Garis putus-putus merah merepresentasikan prediksi yang sempurna (di mana nilai aktual = nilai prediksi). Dapat diamati bahwa titik-titik data membentuk sebuah klaster yang sangat rapat dan terdistribusi secara linear di sepanjang garis diagonal ini. Hal ini membuktikan dua hal penting: (1) Terdapat korelasi yang sangat kuat antara nilai prediksi dan aktual. (2) Model memiliki performa yang konsisten dan tidak bias di seluruh rentang SoC, baik pada kondisi SoC rendah, sedang, maupun tinggi.

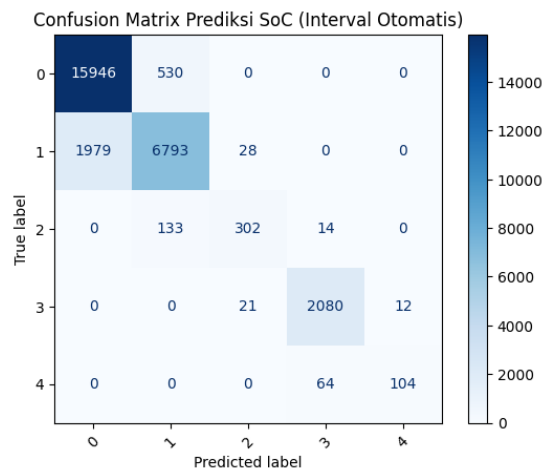
B. Kinerja sebagai Model Klasifikasi

Selain prediksi nilai pasti, kemampuan model untuk mengklasifikasikan status baterai ke dalam rentang tertentu juga sangat penting. Tabel II menunjukkan kinerja model dalam tugas ini.

Tabel II
HASIL KINERJA KLASIFIKASI (5 KELAS INTERVAL SOC)

Metrik	Nilai
Akurasi	0.9007
Presisi (rata-rata makro)	0.9015
Recall (rata-rata makro)	0.9007
F1-Score (rata-rata makro)	0.8980

Dengan akurasi 90.07% dan F1-Score hampir 0.90, model menunjukkan kemampuan yang sangat baik dalam mengkategorikan rentang SoC. Untuk memahami di mana model membuat kesalahan, Gambar 5 menyajikan *confusion matrix*.



Gambar 5. Confusion Matrix untuk Prediksi Klasifikasi SoC (5 Kelas). Diagonal yang dominan menunjukkan tingkat klasifikasi yang benar sangat tinggi.

Dari matriks tersebut, terlihat bahwa sebagian besar prediksi berada di sepanjang diagonal utama (prediksi benar). Kesalahan klasifikasi yang terjadi umumnya hanya terjadi pada kelas-kelas yang berdekatan, yang merupakan kesalahan yang dapat ditoleransi dalam skenario praktis.

IV. KESIMPULAN

Penelitian ini berhasil mendemonstrasikan bahwa sebuah arsitektur hybrid CNN-LSTM yang ringkas mampu memberikan estimasi State of Charge (SoC) dengan kinerja yang sangat tinggi dari dua perspektif: regresi dan klasifikasi. Pada tugas regresi, model mencapai akurasi luar biasa dengan R^2 0.9870. Pada saat yang sama, model ini juga unggul sebagai pengklasifikasi dengan akurasi 90.07% dalam menentukan rentang SoC yang benar. Keberhasilan ganda ini, yang dicapai dengan model yang efisien secara komputasi, menyoroti potensi besar dari pendekatan ini untuk implementasi pada perangkat keras BMS di dunia nyata.

PUSTAKA

- [1] J. Tian, C. Chen, W. Shen, F. Sun, and R. Xiong, "Deep Learning Framework for Lithium-ion Battery State of Charge Estimation: Recent Advances and Future Perspectives," *Energy Storage Materials*, vol. 61, p. 102883, 2023.
- [2] A. P. Renold and N. S. Kathayat, "Comprehensive Review of Machine Learning, Deep Learning, and Digital Twin Data-Driven Approaches in Battery Health Prediction of Electric Vehicles," *IEEE Access*, 2024.
- [3] S. Guo and L. Ma, "A comparative study of different deep learning algorithms for lithium-ion batteries on state-of-charge estimation," *Energy*, vol. 263, p. 125872, 2023.
- [4] K.-H. Kim, K.-H. Oh, H.-S. Ahn, and H.-D. Choi, "Time-Frequency Domain Deep Convolutional Neural Network for Li-Ion Battery SoC Estimation," *IEEE Transactions on Power Electronics*, vol. 39, no. 1, pp. 125–137, 2024.
- [5] I. Oyewole, A. Chehade, and Y. Kim, "A controllable deep transfer learning network with multiple domain adaptation for battery state-of-charge estimation," *Applied Energy*, vol. 312, p. 118726, 2022.
- [6] M. A. Hannan, D. N. T. How, M. S. H. Lipu, P. J. Ker, Z. Y. Dong, M. Mansur, and F. Blaabjerg, "SOC Estimation of Li-ion Batteries With Learning Rate-Optimized Deep Fully Convolutional Network," *IEEE Transactions on Power Electronics*, vol. 36, no. 7, pp. 7349–7353, 2021.
- [7] M. A. Hannan, D. N. T. How, M. S. H. Lipu, et al., "Deep learning approach towards accurate state of charge estimation for lithium-ion batteries using self-supervised transformer model," *Scientific Reports*, vol. 11, no. 1, p. 19541, 2021.
- [8] C. Bian, H. He, and S. Yang, "Stacked bidirectional long short-term memory networks for state-of-charge estimation of lithium-ion batteries," *Energy*, vol. 191, p. 116538, 2020.

LAMPIRAN A

KODE PROGRAM PYTHON MENGGUNAKNA JUPYTER LAB

```
1 # =====
2 # ? Import Library
3 # =====
4
5 import numpy as np
6 import pandas as pd
7 import matplotlib.pyplot as plt
8 from sklearn.model_selection import train_test_split
9 from sklearn.preprocessing import MinMaxScaler
10 from sklearn.metrics import mean_squared_error, r2_score
11
12 import tensorflow as tf
13 from tensorflow.keras.models import Sequential
14 from tensorflow.keras.layers import Conv1D, LSTM, Dense, Dropout, Flatten, Input
15 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

Listing 1. Import Library

```
1 # =====
2 # ? Membaca Dataset Excel
3 # =====
4
5 file_path = "Book45.xlsx" # ganti jika nama file berbeda
6 df = pd.read_excel(file_path)
7
8 print("Jumlah baris dan kolom:", df.shape)
9 display(df.head())
```

Listing 2. Membaca Dataset

```
1 # =====
2 # ? Pra-pemrosesan Data
3 # =====
4
5 # Memilih fitur dan target
6 features = ['voltage', 'current', 'temp 1', 'temp 2', 'temp 3', 'temp 4']
7 target = 'SOC'
```

```

8 data = df[features + [target]]
9
10 # Normalisasi data
11 scaler = MinMaxScaler()
12 data_scaled = scaler.fit_transform(data)
13
14 # Fungsi untuk membuat sekuens
15 def create_sequences(data, n_steps):
16     X, y = [], []
17     for i in range(len(data) - n_steps):
18         # Input: n_steps data sebelumnya (semua fitur)
19         X.append(data[i:i+n_steps, :-1])
20         # Output: 1 data setelahnya (hanya target)
21         y.append(data[i+n_steps, -1])
22     return np.array(X), np.array(y)
23
24 # Membuat sekuens dengan panjang 10
25 n_steps = 10
26 X, y = create_sequences(data_scaled, n_steps)
27
28 # Pembagian data (80% train, 20% test)
29 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

Listing 3. Pra-pemrosesan Data

```

1 # =====
2 # ? Membangun Model CNN-LSTM
3 # =====
4
5 model = Sequential([
6     Input(shape=(X_train.shape[1], X_train.shape[2])), # Input layer
7     Conv1D(filters=64, kernel_size=3, activation='relu'),
8     LSTM(50, activation='relu'),
9     Dropout(0.2),
10    Dense(1)
11 ])
12
13 model.compile(optimizer='adam', loss='mean_squared_error')
14 model.summary()

```

Listing 4. Membangun Model CNN-LSTM

```

1 # =====
2 # ? Melatih Model
3 # =====
4
5 # Callbacks untuk mengoptimalkan pelatihan
6 early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
7 reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=0.0001)
8
9 history = model.fit(
10     X_train, y_train,
11     epochs=100,
12     batch_size=32,
13     validation_split=0.2,
14     callbacks=[early_stopping, reduce_lr],
15     verbose=1
16 )

```

Listing 5. Melatih Model

```

1 # =====
2 # ? Evaluasi Model
3 # =====
4
5 # Prediksi pada data uji
6 y_pred_scaled = model.predict(X_test)
7
8 # Kembalikan hasil prediksi dan data uji ke skala asli
9 target_scaler = MinMaxScaler()
10 target_scaler.min_, target_scaler.scale_ = scaler.min_[-1], scaler.scale_[-1]
11 y_pred = target_scaler.inverse_transform(y_pred_scaled)
12 y_true = target_scaler.inverse_transform(y_test.reshape(-1, 1))
13

```

```

14 # Hitung metrik evaluasi
15 mse = mean_squared_error(y_true, y_pred)
16 r2 = r2_score(y_true, y_pred)
17
18 print("Metrik Evaluasi Prediksi SoC:")
19 print(f"Mean Squared Error (MSE): {mse}")
20 print(f"R-squared (R2) Score: {r2}")

```

Listing 6. Evaluasi Model (Regresi)

```

1 # =====
2 # ? Plot Training & Validation Loss
3 # =====
4
5 plt.figure(figsize=(10, 6))
6 plt.plot(history.history['loss'], label='Training Loss')
7 plt.plot(history.history['val_loss'], label='Validation Loss')
8 plt.title('Training & Validation Loss')
9 plt.xlabel('Epoch')
10 plt.ylabel('Loss')
11 plt.legend()
12 plt.grid(True)
13 plt.show()
14
15 # =====
16 # ? Plot Prediksi vs Aktual SoC (Time Series)
17 # =====
18
19 plt.figure(figsize=(15, 6))
20 plt.plot(y_true, label='Actual SoC')
21 plt.plot(y_pred, label='Predicted SoC', alpha=0.8)
22 plt.title('Perbandingan SoC Aktual vs Prediksi')
23 plt.xlabel('Sample')
24 plt.ylabel('SoC (%)')
25 plt.legend()
26 plt.grid(True)
27 plt.show()
28
29 # =====
30 # ? Plot Prediksi vs Aktual SoC (Scatter Plot)
31 # =====
32
33 plt.figure(figsize=(6,6))
34 plt.scatter(y_true, y_pred, alpha=0.5)
35 plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'r--')
36 plt.xlabel("Actual SoC")
37 plt.ylabel("Predicted SoC")
38 plt.title("Prediksi vs Aktual SoC")
39 plt.grid(True)
40 plt.show()

```

Listing 7. Plotting Hasil